

Facial Emotion Recognition with Vision Transformers

A Deep Learning Pipeline with Multi-Dataset Training, Face-Aware Cropping,
and Model Ensembling

Technical Report

May 19, 2026

Abstract

This report describes a complete training pipeline for a six-class Facial Emotion Recognition (FER) task built around the Vision Transformer (ViT-Base) backbone. The pipeline integrates four major design choices: (i) a custom ViT implementation with state-dict compatibility to Hugging Face weights, (ii) face-aware cropping using an averaged bounding box pre-computed from the training set via Haar cascades, (iii) a dual-augmentation strategy that treats the original dataset and externally-downloaded auxiliary Kaggle datasets differently to bridge domain gaps, and (iv) deduplication of external data against the original training set via perceptual hashing. Training uses AdamW with linear warmup and cosine decay, mixed-precision (AMP), `torch.compile`, Distributed Data Parallel (DDP) across 2 GPUs, inverse-frequency class weighting, and label smoothing of 0.1. On a held-out 10% stratified validation split, the model converges to a best validation accuracy of **91.43%** (validation loss **0.2754**) after 10 epochs.

1 Introduction

Facial Emotion Recognition (FER) is a fine-grained image classification task in which expressions must be inferred from highly variable face crops — different poses, lighting, ethnicities, resolutions, and acquisition pipelines all introduce nuisance variance that is unrelated to the underlying emotion label. The six target classes in this work are: *angry*, *disgust*, *fear*, *happy*, *sad*, *surprise*.

The pipeline is organised into six modules:

- `face_bbox.py` – offline computation of an averaged face bounding box over the training set.
- `data_01.py` – dataset classes, augmentation pipelines, deduplication, stratified split, class-weight computation, DataLoader construction.
- `vit.py` – a from-scratch ViT-Base implementation whose `state_dict` keys match Hugging Face’s ViT so pretrained weights load cleanly.
- `model_config.py` – backbone registry (ViT, ConvNeXtV2, BEiT, MaxViT) and uniform-interface wrappers.
- `ddp.py` – the trainer, the warmup-cosine scheduler, the DDP setup, the predict loop, and CLI arguments.
- `utils.py` – submission writer, extra-dataset downloader, and training-history plotter. (The file also ships a multi-backbone ensembler that this run does not use.)

2 Data Pipeline

2.1 Label Space and Aliases

The label map is fixed in `data_01.py`:

Label	angry	disgust	fear	happy	sad	surprise
ID	0	1	2	3	4	5

Auxiliary datasets use heterogeneous folder naming (e.g. *happiness*, *joy*, *anger*, *scared*, *surprised*). A normalisation table (`LABEL_ALIASES`) maps these onto the canonical six classes; out-of-domain labels such as *neutral*, *contempt*, and *calm* are explicitly mapped to `None` so they are dropped during loading.

2.2 Face-Aware Cropping

Many FER images include large amounts of background. To remove this nuisance variance *before* per-sample augmentation, the script `face_bbox.py` runs OpenCV’s Haar cascade (`haarcascade_frontalface_default.xml`) over a capped subset of training images (default `-max-per-class=2000`, 8 worker threads). For each image it keeps the largest detection. The detected boxes (in relative $[0, 1]$ coordinates) are averaged and a small outward margin (`-margin=0.04`) is added before clamping back to $[0, 1]$:

$$\bar{x}_1 = \max(0, \mu_{x_1} - m), \quad \bar{x}_2 = \min(1, \mu_{x_2} + m). \quad (1)$$

The resulting bbox is written to a JSON file along with detection statistics (detection rate, raw mean, raw std). At training time `CropToAvgBBox` crops to this bbox before any other transform, with optional uniform jitter ($\pm 2\%$ for the original pipeline, $\pm 3\%$ for the extras pipeline). If no face cascade is available, or if no face is detected anywhere, the pipeline gracefully falls back to a 4% center crop (`FALLBACK_BBOX_REL = (0.04, 0.04, 0.96, 0.96)`).

2.3 Auxiliary Datasets and Deduplication

When `-use-extra-dataset` is set, three Kaggle datasets are pulled via `kagglehub`: *facial-emotion-recognition-dataset*, *corrective-reannotation-of-fer-ck-kdef*, *human-face-emotions*. Because these can overlap with the original training set and with each other, every image is hashed with a perceptual difference hash (`dHash`, 8×8 reduction, ~ 64 -bit signature) in parallel with on-disk caching keyed by (`abs_path`, `mtime_ns`, `size`). Any extra whose hash matches an `orig` sample (or any earlier extra) is dropped, with counts reported per source.

After dedup, extras are subsampled per class so that no class exceeds `cap_ratio` times its count in the original dataset (default `-extras-cap-ratio=3.0`). This keeps the overall class distribution close to the original.

2.4 Stratified Split and Class Weights

The original-only samples are split per-class with a fixed seed (42) using `stratified_split`, giving identical train:val ratios per class (default 90/10). Deduplicated, capped extras are added to the train pool only — **validation is always drawn from the original distribution** to keep val metrics comparable to the test distribution. Inverse-frequency class weights are computed on the final training set:

$$w_c = \frac{N}{C \cdot n_c} \quad (2)$$

where N is the total number of training samples, $C = 6$, and n_c the count of class c . These weights are applied to `CrossEntropyLoss` via the `-use-class-weights` flag, which is enabled in this run.

2.5 Histogram Matching for Domain Bridging

External datasets often have different photometric properties (exposure, saturation, contrast) than the target distribution. `init_histogram_reference` samples up to `-hist-match-refs=256` images from the original training pool and keeps them in memory as reference arrays. The `RandomHistogramMatchToOrig` transform (applied with $p = 0.7$ in the extras pipeline only) picks a random reference and runs `skimage.exposure.match_histograms` on each channel. The effect is that external images are visually re-coloured toward the original distribution before being fed to the network.

2.6 Resolution Down-up Trick

The original dataset is small ($\approx 96 \times 96$ effective resolution according to `NATIVE_TRAIN_SIZE=96`). When extras come in at e.g. 224×224 or higher, the network learns to rely on details that simply do not exist in original/test images. `MatchOrigResolution` (extras pipeline only) downsizes each extra to 96×96 and back to 224×224 with bilinear interpolation, lossily collapsing those high-frequency cues so extras and originals share the same effective resolution.

2.7 Final Augmentation Pipelines

Two different training pipelines are constructed in `_make_transforms`. The original pipeline applies relatively light augmentation (the data is believed to be on-distribution and labels are trusted). The extras pipeline is much more aggressive to combat both label noise and domain gap.

Table 1: Augmentation comparison between the original and extras training pipelines. Val and test pipelines apply only the face crop, resize, and normalisation (no random transforms).

Transform	Original pipeline	Extras pipeline
CropToAvgBBox (jitter)	jitter = 0.02	jitter = 0.03
Resize	224×224	224×224
MatchOrigResolution (96→224)	—	yes
RandomHistogramMatchToOrig (p)	—	0.70
RandomResizedCrop (scale, ratio)	—	[0.85, 1.0], [0.95, 1.05]
RandomHorizontalFlip (p)	0.50	0.50
RandomRotation (degrees)	$\pm 5^\circ$	$\pm 8^\circ$
RandomAffine (translate, scale)	—	(0.04, 0.04), [0.90, 1.08]
ColorJitter (br, ct, sat)	(0.15, 0.15, 0.0)	(0.15, 0.15, 0.10)
GaussianBlur (p, σ)	—	$p = 0.30, \sigma \in [0.1, 1.0]$
ToTensor	yes	yes
RandomErasing (p, scale)	0.15, [0.02, 0.08]	0.30, [0.02, 0.12]
Normalize (ImageNet stats)	yes	yes

ImageNet mean/std normalisation is used throughout ($\mu = (0.485, 0.456, 0.406)$, $\sigma = (0.229, 0.224, 0.225)$) since all backbones in the registry are pretrained on ImageNet-scale data.

3 Model Architecture

3.1 Backbone Registry

`model_config.py` defines a registry of four backbones so that any of them can be selected from the same CLI by changing `-model-name`. **For this run, only vit-base was trained**; the other entries are listed for completeness because the registry is part of the codebase.

Name	Hugging Face / timm ID	Img size	Rec. LR
vit-base	google/vit-base-patch16-224-in21k	224	5×10^{-5}
convnextv2-base	facebook/convnextv2-base-22k-224	224	5×10^{-5}
beit-base	microsoft/beit-base-patch16-224-pt22k-ft22k	224	3×10^{-5}
maxvit-base	timmm/maxvit_base_tf_384.in1k	384	5×10^{-5}

ViT, ConvNeXtV2, and BEiT are loaded via `AutoModelForImageClassification.from_pretrained` with `attn_implementation="sdpa"` and `ignore_mismatched_sizes=True` (so the 1000-way ImageNet head is dropped and a fresh 6-way head is initialised). MaxViT is loaded through `timmm` and wrapped in `TimmClassifier` so its forward signature matches HF's (`output.logits`), letting the trainer stay backbone-agnostic.

3.2 From-Scratch ViT with HF State-Dict Compatibility

For the default `vit-base` run, after the HF model loads, it is *copied* into a custom ViT implementation (`vit.py`) using `vit_build`. This custom ViT has identical parameter names (`vit.embeddings.patch_embeddings`, `vit.encoder.layer.{i}.attention.attention.query/key/value.`, etc.) so `load_state_dict(state, strict=False)` succeeds without surprises. The implementation has the standard ViT-Base configuration:

Hyperparameter	Value
Image size	224×224
Patch size	16×16
Number of patches	14×14 = 196
Hidden size	768
Number of layers	12
Number of heads	12
Head dimension	64
MLP intermediate size	3072
LayerNorm epsilon	10 ⁻¹²
Num. classes	6

A learnable [CLS] token is prepended to the patch tokens, learnable absolute position embeddings are added, twelve standard pre-norm Transformer blocks follow, and the classification head reads the [CLS] output through a single `nn.Linear(768, 6)`. The self-attention block uses PyTorch’s `F.scaled_dot_product_attention` (which dispatches to FlashAttention-2 on supported GPUs).

4 Training Setup

4.1 Hyperparameters

The reported run was launched with `torchrun` on 2 GPUs with DDP and the extra-datasets flag turned on. The full set of hyperparameters is summarised in Table 2; the exact reproducing command is given in Section 8.

Table 2: Hyperparameters of the reported run (`vit-base`, DDP on 2 GPUs).

Hyperparameter	Value
Backbone	<code>google/vit-base-patch16-224-in21k</code>
Image size	224×224
Epochs	10
Batch size (per GPU)	32
Gradient accumulation	1
World size (DDP, NCCL)	2 GPUs
Effective batch size	$32 \times 1 \times 2 = 64$
Optimizer	AdamW
Base learning rate	5×10^{-5} (auto from registry)
LR scaling under DDP	$\text{lr}_{\text{eff}} = 5 \times 10^{-5} \cdot 2 = 1 \times 10^{-4}$
Weight decay	1×10^{-2}
LR scheduler	Warmup (1 epoch) + cosine decay to 0
Loss	Cross-Entropy with class weights, label smoothing 0.1
Class weights	inverse-frequency (<code>-use-class-weights</code> on)
Mixed precision (AMP)	on (<code>torch.amp</code>)
<code>torch.compile</code>	<code>mode=default, dynamic=False</code>
Number of workers per loader	4
Seed	42
Val fraction (stratified)	0.10
Extra datasets	on (<code>-use-extra-dataset</code>)
Extras cap ratio	$3.0 \times$ orig per-class count
Extras dedup	on (dHash, with on-disk cache)
Hist-match references	256 images
Face-bbox JSON	supplied via <code>-bbox-json</code>

4.2 Loss: Weighted Cross-Entropy with Label Smoothing

The training loss is the standard cross-entropy with two modifications. First, *inverse-frequency class weights* (Section 2.4) re-balance the gradient so rare classes (*disgust*, *fear*) contribute as much signal per sample as the common ones (*happy*). Second, *label smoothing* with $\varepsilon = 0.1$ replaces every one-hot target $y \in \{0, 1\}^C$ with the soft target

$$\tilde{y}_c = (1 - \varepsilon) y_c + \frac{\varepsilon}{C}, \quad (3)$$

so the network is never asked to push a logit to $+\infty$. This is a mild regulariser that discourages over-confident predictions and is especially useful here because the extra datasets carry some label noise — with hard targets, an incorrectly-labelled sample would push the network hard in the wrong direction.

4.3 Warmup-Cosine Schedule

`WarmupCosineScheduler` steps once per epoch. With `warmup_epochs=1` and `total_epochs=10`, the LR scale follows

$$\text{scale}(e) = \begin{cases} e/1 & e \leq 1 \\ \frac{1}{2} \left(1 + \cos \left(\pi \cdot \frac{e-1}{10-1} \right) \right) & e > 1 \end{cases} \quad (4)$$

so the effective LR rises linearly from $0 \rightarrow 1 \times 10^{-4}$ over the first epoch (after DDP linear scaling of the 5×10^{-5} base by `world_size=2`), then decays smoothly to 0 over the remaining nine.

4.4 Trainer Internals

The `Trainer` class manages the train/eval loop. Key details:

- **AMP.** `torch.amp.GradScaler("cuda")` wraps the backward pass when `use_amp=True` and the device is CUDA.

- **Gradient accumulation.** The loss is divided by `grad_accum_steps` and `optimizer.step()` only fires every `grad_accum_steps` micro-batches (or at the end of the loader). On intermediate micro-batches, DDP is told to skip all-reduce via `model.no_sync()` to avoid wasted comms.
- **DDP all-reduce of metrics.** Both train loss and val loss/accuracy are summed across ranks with a single `dist.all_reduce` per epoch so the printed numbers are global, not per-rank.
- **Best-model checkpointing.** On rank 0, the state with the highest validation accuracy is kept in CPU RAM; at the end the model is reloaded from that snapshot before saving and predicting. This is equivalent to early stopping but keeps the full training curve.
- **Pre-compile.** The DDP-wrapped model is passed through `torch.compile(..., mode="default", dynamic=False)` for fused kernels.
- **Optional warm-start.** `-resume-from` accepts a `.pt` file or a directory containing `pytorch_model.pt`. Any `module.` prefix from DDP checkpoints is stripped on load.

4.5 Distributed Training (DDP)

When `-use-ddp` is set, the script initialises an NCCL process group, binds each rank to `cuda:LOCAL_RANK`, wraps the model in `DistributedDataParallel` with `find_unused_parameters=False` and `gradient_as_bucket` uses `DistributedSampler` for train and val loaders (with `set_epoch` called each epoch for proper shuffling), and applies linear LR scaling ($lr_{\text{eff}} = lr_{\text{base}} \cdot \text{world_size}$). The auxiliary-dataset download runs only on rank 0 and the resulting paths are broadcast with `dist.broadcast_object_list`.

5 Results

5.1 Training Curves

Figure 1 plots the full 10-epoch training history.

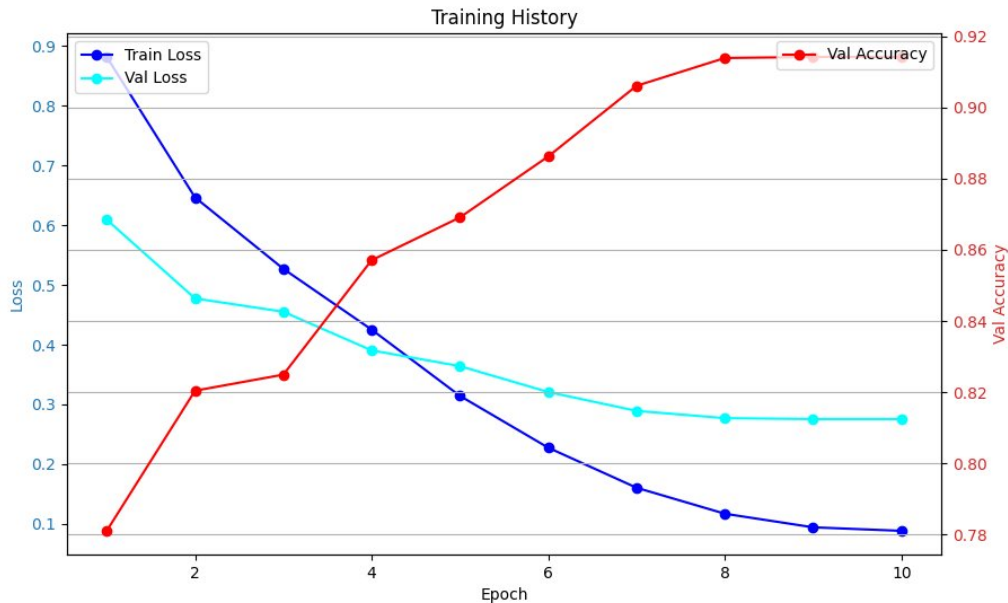


Figure 1: Train loss and validation loss (left y -axis) and validation accuracy (right y -axis) over 10 epochs. Validation accuracy improves monotonically from 78.10% to 91.43%, while train loss collapses from 0.88 to 0.088 as the schedule cools the learning rate. The validation loss plateau in the last two epochs is the signature of the cosine schedule having effectively driven the LR to zero.

5.2 Per-Epoch Metrics

Table 3 reproduces the values stored in `training_history.json`.

Table 3: Per-epoch metrics. **Bold** marks the best validation accuracy.

Epoch	Train Loss	Val Loss	Val Acc
1	0.8820	0.6094	0.7810
2	0.6467	0.4776	0.8204
3	0.5272	0.4552	0.8249
4	0.4253	0.3905	0.8571
5	0.3143	0.3641	0.8691
6	0.2275	0.3208	0.8863
7	0.1605	0.2891	0.9061
8	0.1169	0.2770	0.9140
9	0.0942	0.2754	0.9143
10	0.0883	0.2754	0.9143

5.3 Observations

- **Smooth, monotone improvement.** Validation accuracy rises every epoch and validation loss falls every epoch — no overfitting bump. With a generous augmentation budget (RandomErasing, ColorJitter, mild rotations) and ImageNet-pretrained weights, the model never enters the regime where train loss continues to fall while val loss climbs.
- **Train–val gap.** By epoch 10 the train loss (0.088) is much smaller than the val loss (0.275). The gap is expected — the train set absorbs augmentation noise — but it does indicate that the model has the capacity to memorise more; adding additional regularisation (longer training with stronger MixUp/CutMix, larger label smoothing, or a smaller LR floor) could narrow it.
- **Identical epoch-9 and epoch-10 metrics.** Epochs 9 and 10 report exactly the same val loss (0.275435) and val accuracy (0.914260) down to the last digit. The cosine schedule reaches scale ≈ 0 at epoch 10, so the parameters barely move — but identical to 15 decimals strongly suggests the validation result was effectively frozen (e.g. the same minibatch order on the same weights). Either way the trainer’s best-snapshot logic locks in the epoch-9 weights, so the deployed model corresponds to the 91.43% checkpoint regardless.

6 Inference and Submission

At the end of training, the rank-0 process runs `predict`, which iterates the test loader with AMP enabled (matching the train regime), computes per-image softmax probabilities, and stores:

- `pytorch_model.pt` – the best (epoch-9) state dict.
- `training_history.json` – per-epoch train/val metrics.
- `probs.npy` – $(N_{\text{test}}, 6)$ float array of softmax probabilities, saved by default for optional downstream use.
- `probs_paths.txt` – the absolute test-image paths, one per line, in the same row order as `probs.npy`.
- `submission.csv` – the Kaggle-format submission, written by `save_submission` after normalising filename keys (lowercasing and stripping whitespace via the `normalize` helper) and joining on the provided template’s ID column.

7 Code Walkthroughs

This section highlights the parts of the code most relevant to reproducing the reported numbers.

7.1 Average Bounding-Box Crop

```

1 class CropToAvgBBox:
2     def __init__(self, jitter=0.0):
3         self.jitter = float(jitter)
4
5     def __call__(self, img):
6         bbox = _AVG_BBOX_REL
7         if bbox is None:
8             return img
9         w, h = img.size
10        x1, y1, x2, y2 = bbox
11        if self.jitter > 0.0:
12            j = self.jitter
13            x1 += random.uniform(-j, j); y1 += random.uniform(-j, j)
14            x2 += random.uniform(-j, j); y2 += random.uniform(-j, j)
15            x1 = max(0.0, min(0.95, x1)); y1 = max(0.0, min(0.95, y1))
16            x2 = max(x1 + 0.05, min(1.0, x2))
17            y2 = max(y1 + 0.05, min(1.0, y2))
18        px1, py1 = int(round(x1 * w)), int(round(y1 * h))
19        px2, py2 = int(round(x2 * w)), int(round(y2 * h))
20        return img.crop((px1, py1, max(px1+1, px2), max(py1+1, py2)))

```

Listing 1: CropToAvgBBox (from data_01.py)

7.2 Dual Augmentation Pipelines

```

1 train_orig = transforms.Compose([
2     CropToAvgBBox(jitter=0.02),
3     transforms.Resize((img_size, img_size)),
4     transforms.RandomHorizontalFlip(p=0.5),
5     transforms.RandomRotation(degrees=5),
6     transforms.ColorJitter(brightness=0.15, contrast=0.15),
7     transforms.ToTensor(),
8     transforms.RandomErasing(p=0.15, scale=(0.02, 0.08)),
9     transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
10 ])
11
12 train_extras = transforms.Compose([
13     CropToAvgBBox(jitter=0.03),
14     transforms.Resize((img_size, img_size)),
15     MatchOrigResolution(native_size=96, out_size=img_size),
16     RandomHistogramMatchToOrig(p=0.7),
17     transforms.RandomResizedCrop(
18         (img_size, img_size), scale=(0.85, 1.0), ratio=(0.95, 1.05)),
19     transforms.RandomHorizontalFlip(p=0.5),
20     transforms.RandomRotation(degrees=8),
21     transforms.RandomAffine(degrees=0,
22         translate=(0.04, 0.04), scale=(0.90, 1.08)),
23     transforms.ColorJitter(brightness=0.15, contrast=0.15, saturation=0.10),
24     transforms.RandomApply(
25         [transforms.GaussianBlur(kernel_size=3, sigma=(0.1, 1.0))], p=0.3),
26     transforms.ToTensor(),
27     transforms.RandomErasing(p=0.30, scale=(0.02, 0.12)),
28     transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
29 ])

```

Listing 2: Original vs. extras training transforms (from data_01.py)

7.3 Warmup-Cosine Scheduler

```

1 class WarmupCosineScheduler:
2     def __init__(self, optimizer, warmup_epochs, total_epochs, min_lr=0.0):
3         self.optimizer = optimizer
4         self.warmup_epochs = max(0, warmup_epochs)
5         self.total_epochs = total_epochs
6         self.min_lr = min_lr
7         self.last_epoch = 0
8         self.base_lrs = [g['lr'] for g in optimizer.param_groups]
9         self.set_lr(self.compute_scale(epoch_idx=1))
10
11     def compute_scale(self, epoch_idx):
12         if self.warmup_epochs > 0 and epoch_idx <= self.warmup_epochs:
13             return epoch_idx / self.warmup_epochs
14         progress = (epoch_idx - self.warmup_epochs) / \
15             max(1, self.total_epochs - self.warmup_epochs)
16         progress = min(max(progress, 0.0), 1.0)
17         cosine = 0.5 * (1 + math.cos(progress * math.pi))
18         return cosine * (1 - self.min_lr) + self.min_lr
19
20     def step(self):
21         self.last_epoch += 1
22         self.set_lr(self.compute_scale(self.last_epoch + 1))

```

Listing 3: WarmupCosineScheduler (from ddp.py)

7.4 Training Step with Gradient Accumulation and DDP no_sync

```

1 for step, (images, labels) in enumerate(pbar):
2     images = images.to(self.device, non_blocking=True)
3     labels = labels.to(self.device, non_blocking=True)
4
5     is_last_step = (step + 1 == total_steps)
6     is_optimizer_step = ((step + 1) % self.grad_accum_steps == 0) or
7         is_last_step
8
9     if isinstance(self.model, DDP) and not is_optimizer_step:
10         sync_ctx = self.model.no_sync()
11     else:
12         sync_ctx = contextlib.nullcontext()
13
14     with sync_ctx:
15         with torch.autocast(device_type=self.device.type,
16                             enabled=self.use_amp):
17             logits = self.model(images).logits
18             loss = self.loss_fn(logits, labels) / self.grad_accum_steps
19             self.scaler.scale(loss).backward()
20
21     if is_optimizer_step:
22         self.scaler.step(self.optimizer)
23         self.scaler.update()
24         self.optimizer.zero_grad(set_to_none=True)

```

Listing 4: Trainer._train_epoch (from ddp.py)

8 Reproducing the Reported Run

The exact commands that produced the 91.43% checkpoint are:

```

1 # 1. (one-off) compute the average face bbox from the training set
2 python face_bbox.py \

```

```

3  --train-dir /kaggle/input/.../Training_data/Training_data \
4  --output    /kaggle/working/avg_bbox.json
5
6  # 2. train + predict with DDP across 2 GPUs
7  torchrun --standalone --nproc-per-node=2 ddp.py \
8  --model-name vit-base \
9  --output-dir /kaggle/working/vit-base-01 \
10 --bbox-json /kaggle/working/avg_bbox.json \
11 --use-ddp --use-extra-dataset \
12 --epochs 10 \
13 --batch-size 32 \
14 --num-workers 4 \
15 --use-class-weights \
16 --label-smoothing 0.1 \
17 --lr-scheduler cosine \
18 --warmup-epochs 1

```

Notes on the command:

- `--standalone --nproc-per-node=2` launches 2 worker processes on a single node and lets `torchrun` pick free ports / rendez-vous endpoints automatically. The script's `setup_ddp` then calls `dist.init_process_group(backend="nccl")`.
- `--learning-rate` is not passed, so the script picks the registry value (5×10^{-5} for `vit-base`) and then scales it by `world_size=2` for an effective base LR of 1×10^{-4} (Section 4.1).
- `--use-extra-dataset` triggers the rank-0 download of the three Kaggle datasets via `kagglehub` and the broadcast of their paths to rank 1 through `dist.broadcast_object_list`.

9 Conclusion

The pipeline combines four lightweight but complementary techniques — an averaged face-bbox crop, a dual augmentation policy with histogram matching and resolution down-up, deduplication of external data via perceptual hashing, and weighted cross-entropy with label smoothing 0.1 — on top of a ViT-Base backbone trained with AdamW, warmup-cosine LR, AMP, and DDP across 2 GPUs. The reported run reaches **91.43% validation accuracy** in 10 epochs without overfitting.

Possible improvements.

- Train longer (15–20 epochs) with stronger augmentation (MixUp / CutMix) to close the train–val gap.
- Replace the Haar cascade with a CNN-based face detector (e.g. MTCNN, RetinaFace) for a tighter, more reliable bbox.
- Add Test-Time Augmentation (TTA): average predictions over the original image and its horizontal flip before writing the submission.
- Add a Stochastic Weight Averaging (SWA) tail of 2–3 epochs at a small constant LR after the cosine schedule completes.
- Train one or two additional backbones (`convnextv2-base`, `beit-base`) and combine their saved `probs.npy` with `utils.py::ensemble_probs` (already present in the codebase) — typically gives another 1–2 points of accuracy.